

LAPORAN PRAKTIKUM
MATA KULIAH PEMROGRAMAN WEB
Modul 10: Javascript Lanjutan



Oleh:
Davin Aileenskie Sofiandy
707012500135
D4SIKC 49-04

PROGRAM STUDI S.Ter SISTEM INFORMASI KOTA CERDAS
FAKULTAS ILMU TERAPAN
UNIVERSITAS TELKOM
BANDUNG
2026

2. Tujuan Praktikum

- Memahami konsep dasar JavaScript sebagai bahasa pemrograman web modern.
- Menggunakan fitur Modern JavaScript ES6+ seperti module, let/const, arrow function, template literal, spread operator, dan array method.
- Memahami konsep state, immutability, dan functional programming sebagai dasar React.
- Membangun aplikasi webstore sederhana berbasis state menggunakan Vanilla JavaScript.

3. Alat dan Bahan

No	Alat/Bahan	Keterangan
1	Laptop/PC	Digunakan untuk membuat dan menjalankan program.
2	Visual Studio Code	Digunakan sebagai code editor.
3	Web Browser	Digunakan untuk menjalankan dan melihat hasil webstore.
4	HTML, CSS, JavaScript	Digunakan untuk membuat struktur, tampilan, dan logika program.
5	jQuery CDN	Digunakan pada fitur newsletter.
6	Live Server	Digunakan untuk menjalankan aplikasi web secara lokal.

4. Langkah-Langkah Praktikum

Langkah praktikum disusun berdasarkan perintah pada modul, yaitu membuat folder webstore, mengisi file program, lalu menjalankan hasilnya melalui browser.

1. Membuat folder proyek bernama webstore.
2. Membuat struktur file yang terdiri dari index.html, style.css, app.js, folder api, dan folder components.
3. Mengisi index.html sebagai halaman utama webstore dan menghubungkan style.css, jQuery CDN, serta app.js dengan type module.
4. Mengisi style.css untuk mengatur tampilan produk, tombol, layout, cart, newsletter, dan responsive design.
5. Mengisi productApi.js untuk menyediakan data produk secara asynchronous menggunakan Promise.
6. Mengisi ProductList.js untuk menampilkan daftar produk ke halaman web.
7. Mengisi Cart.js untuk menampilkan keranjang belanja, total harga, tombol remove, dan checkout.
8. Mengisi Newsletter.js untuk menampilkan form subscribe dan menangani submit form.
9. Mengisi app.js sebagai pengatur utama state, render, add to cart, remove cart, filter, sort, dan checkout.
10. Menjalankan proyek menggunakan Live Server, kemudian membuka halaman webstore di browser.

Struktur File yang Digunakan

File	Fungsi dalam Program
index.html	Membuat struktur halaman, tombol Filters/Sort, area product-list, dan cart.
style.css	Mengatur desain halaman webstore agar rapi dan responsif.
app.js	Mengatur state aplikasi, render ulang UI, sort, filter, cart, dan checkout.
api/productApi.js	Berisi data produk dan simulasi fetch asynchronous.
components/ProductList.js	Komponen untuk menampilkan kartu produk.
components/Cart.js	Komponen untuk menampilkan isi keranjang dan total belanja.
components/Newsletter.js	Komponen form newsletter dengan JavaScript/jQuery.
Core/store.js	Mengatur state global aplikasi. Mengatur state global aplikasi.
Core/utils.js	Menyediakan fungsi bantu untuk menyalin data.

5. Hasil dan Dokumentasi Program

Hasil dari praktikum ini adalah aplikasi Vanilla JS Web Store yang dapat menampilkan daftar produk dan merespons interaksi pengguna secara dinamis.

Bagian/Fitur	Hasil yang Ditampilkan
Halaman utama	Muncul halaman Electronics dengan breadcrumb, tombol Filters, tombol Sort, daftar produk, dan panel Cart.
Product List	Produk tampil dalam bentuk card berisi gambar, kategori, judul, deskripsi, rating,

	harga, diskon, dan tombol Add.
Add to Cart	Saat tombol Add diklik, produk masuk ke keranjang. Jika produk sama ditambahkan lagi, jumlah/qty bertambah.
Remove Cart	Saat tombol Remove diklik, jumlah produk berkurang. Jika qty habis, produk hilang dari keranjang.
Total Harga	Cart menghitung total dengan reduce() berdasarkan harga dikali qty.
Checkout	Jika cart kosong muncul alert keranjang masih kosong. Jika cart berisi produk, muncul alert checkout berhasil dan cart dikosongkan.
Sort	Produk dapat diurutkan berdasarkan harga naik atau turun.
Filters	Produk dapat difilter untuk menampilkan rating minimal 4.6.
Newsletter	Form email dapat disubmit dan berubah menjadi pesan Thanks for subscribing!.

6. Analisis dan Pembahasan

Program sudah menerapkan konsep Modern JavaScript ES6+ sesuai materi praktikum. Kode dibuat modular dengan import/export sehingga setiap file memiliki tanggung jawab masing-masing.

Konsep Materi	Penerapan pada Program
Module	app.js mengimpor fetchProducts, ProductList, Cart, dan Newsletter dari file berbeda.
State	Variabel state menyimpan kondisi products, cart, sortedAsc, dan filtered.
Immutability	Perubahan cart dan products dilakukan dengan spread operator, map(), dan filter(), bukan mengubah data lama secara langsung.
Arrow Function	Digunakan pada setState, addToCart, removeFromCart, toggleSort, toggleFilter, dan callback event.
Template Literal	Digunakan untuk membuat elemen HTML dinamis pada ProductList, Cart, dan Newsletter.
Functional Programming	Menggunakan find(), map(), filter(), sort(), dan reduce() untuk mengolah data produk dan cart.
Asynchronous JavaScript	fetchProducts menggunakan async function dan Promise untuk simulasi pengambilan data produk.
Event Handling	Tombol Add, Remove, Checkout, Sort, Filter, dan form Newsletter diberi event handler.
jQuery	Newsletter menggunakan jQuery jika tersedia, dan tetap memiliki fallback JavaScript murni jika jQuery tidak tersedia.

Dibandingkan contoh modul, struktur file pada program ini dibuat lebih sederhana karena state dikelola langsung di app.js, bukan dipisahkan ke folder core. Walaupun begitu, konsep yang diminta tetap terpenuhi karena program tetap menggunakan state, module, immutability, component-oriented JavaScript, event handling, dan asynchronous data.

7. Kesimpulan

Berdasarkan praktikum yang dilakukan, aplikasi webstore sederhana berhasil dibuat menggunakan Vanilla JavaScript ES6+. Aplikasi mampu menampilkan produk, mengelola keranjang belanja, melakukan sorting dan filtering, serta menampilkan newsletter. Praktikum ini menunjukkan bahwa konsep JavaScript modern seperti module, state, immutability, arrow function, dan array method sangat penting sebagai dasar pengembangan aplikasi web modern dan fondasi untuk memahami React.

8. Lampiran

Link repositori GitHub : <https://github.com/AriqAhmadNayaka/WebPro4904/tree/DavinAileenskieSofiandy-707012500135>

Lampiran potongan program utama:

productApi.js

```
// Fungsi ini mensimulasikan proses mengambil data produk dari API/server.
export const fetchProducts = async () => {
  return new Promise((resolve) => {
    // setTimeout digunakan untuk memberi efek delay seperti request asynchronous.
    setTimeout(() => {
      // Data produk statis yang akan dikirim ke app.js melalui Promise.
      resolve([
        {
          id: 1,
          title: "Mechanical Keyboard",
          price: 150,
          rating: { rate: 4.8, count: 230 },
          image:
            "https://images.unsplash.com/photo-1618384887929-16ec33fab9ef?auto=format&fit=crop&w=800&q=80",
        },
        {
          id: 2,
          title: "Gaming Mouse",
          price: 80,
          rating: { rate: 4.6, count: 150 },
          image:
            "https://images.unsplash.com/photo-1527814050087-3793815479db?auto=format&fit=crop&w=800&q=80",
        },
        {
          id: 3,
          title: '4K Monitor 24"',
          price: 300,
          rating: { rate: 4.9, count: 85 },
          image:
            "https://images.unsplash.com/photo-1527443224154-c4a3942d3acf?auto=format&fit=crop&w=800&q=80",
        },
        {
          id: 4,
          title: "Premium Coffee Beans",
          price: 25,
          rating: { rate: 4.7, count: 500 },
          image:
```

```

        "https://images.unsplash.com/photo-1559056199-641a0ac8b55e?auto=format&fit=crop&w=800&q=80",
      },
      {
        id: 5,
        title: "Travel Tumbler",
        price: 35,
        rating: { rate: 4.5, count: 120 },
        image:
          "https://images.unsplash.com/photo-1578500494198-246f612d3b3d?auto=format&fit=crop&w=800&q=80",
      },
      {
        id: 6,
        title: "Noise Cancelling Headphones",
        price: 200,
        rating: { rate: 4.8, count: 320 },
        image:
          "https://images.unsplash.com/photo-1505740420928-5e560c06d30e?auto=format&fit=crop&w=800&q=80",
      },
    ]);
  }, 800);
});
};

```

Cart.js

```

// Komponen Cart bertugas menampilkan isi keranjang dan tombol checkout.
export function Cart(cart, onCheckout) {
  const container = document.getElementById("cart");
  container.innerHTML = "<h2>Your Cart</h2>";

  // Jika cart kosong, tampilkan pesan kosong dan hentikan proses render.
  if (cart.length === 0) {
    container.innerHTML += "<p class='empty-msg'>Cart is empty.</p>";
    return;
  }

  // Membuat wrapper untuk daftar produk yang masuk ke keranjang.
  const list = document.createElement("div");
  list.className = "cart-list";

  // Loop setiap item cart dan ubah menjadi elemen HTML.

```

```

    cart.map((item) => {
      const div = document.createElement("div");
      div.className = "cart-item";
      div.innerHTML = `
        
        <div class="cart-details">
          <h4>${item.title}</h4>
          <span>${item.price}</span>
        </div>
      `;
      list.appendChild(div);
    });

    container.appendChild(list);

    // reduce digunakan untuk menghitung total harga seluruh item di cart.
    const total = cart.reduce((acc, item) => acc + (item.price || 0), 0);

    // Footer berisi total harga dan tombol checkout.
    const footer = document.createElement("div");
    footer.className = "cart-footer-summary";
    footer.innerHTML = `
      <div class="total-row">
        <span>Total:</span>
        <span class="total-price">${total.toFixed(2)}</span>
      </div>
      <button class="btn-primary" id="checkout-btn">Buy Now</button>
    `;

    // Event click tombol Buy Now akan menjalankan fungsi checkout dari app.js.
    footer.querySelector("#checkout-btn").addEventListener("click", onCheckout);
    container.appendChild(footer);
  }

```

Newsletter.js

```

// Komponen Newsletter dibuat dengan jQuery untuk form subscribe.
export function Newsletter() {
  // Cegah newsletter dibuat dua kali saat render dipanggil berulang.
  if (document.querySelector(".newsletter-section")) return;

  // Membuat container dan isi newsletter menggunakan sintaks jQuery.
  const $container = $('<div class="newsletter-section"></div>');
  const $content = $(`
    <div class="newsletter-content">
      <h3>Subscribe to our updates</h3>
      <p>Get the latest tech news and offers.</p>

```

```

    <form id="newsletter-form">
      <input type="email" placeholder="Enter your email" required />
      <button type="submit" class="btn-primary">Subscribe</button>
    </form>
    <div class="success-message" style="display:none; margin-top: 10px; color:
green;">
      Thanks for subscribing!
    </div>
  </div>
`);

$container.append($content);
$("#app").append($container);

// Saat form dikirim, halaman tidak refresh dan form diganti pesan sukses.
$content.find("form").on("submit", function (event) {
  event.preventDefault();

  const $form = $(this);

  // Efek slideUp dan fadeIn merupakan contoh penggunaan animasi jQuery.
  $form.slideUp(400, function () {
    $content.find(".success-message").fadeIn();
  });
});
}

```

ProductList.js

```

// Komponen ProductList menampilkan daftar produk ke dalam elemen #product-list.
export function ProductList(products, onAdd) {
  const container = document.getElementById("product-list");

  // Kosongkan container agar data lama tidak menumpuk saat render ulang.
  container.innerHTML = "";

  // map digunakan untuk mengubah setiap object produk menjadi card HTML.
  products.map((product) => {
    const card = document.createElement("div");
    card.className = "product-card";

    // Diskon dibuat random hanya untuk kebutuhan tampilan badge.
    const discount = Math.floor(Math.random() * 20) + 5;

    // Jika produk tidak punya rating, gunakan nilai default.
    const rating = product.rating || { rate: 4.5, count: 100 };
  });
}

```

```

// Template literal memudahkan penulisan struktur HTML product card.
card.innerHTML = `
  <div class="card-image">
    
  </div>
  <div class="card-content">
    <span class="badge">Up to ${discount}% off</span>
    <a href="#" class="card-title" title="${product.title}">${product.title}</a>

    <div class="rating-row">
      <div class="stars">
        ${getStarSVG().repeat(5)}
      </div>
      <span class="rating-score">${rating.rate}</span>
      <span class="rating-count">(${rating.count})</span>
    </div>

    <ul class="features-list">
      <li class="feature">
        <svg xmlns="http://www.w3.org/2000/svg" fill="none" viewBox="0 0 24 24">
          <path stroke="currentColor" stroke-linecap="round" stroke-
linejoin="round" stroke-width="2" d="M13 7h6l2 4m-8-4v8m0-8V6a1 1 0 0 0-1-1H4a1 1 0 0
0-1 1v9h2m8 0H9m4 0h2m4 0h2v-4m0 0h-5m3.5 5.5a2.5 2.5 0 1 1-5 0 2.5 2.5 0 0 1 5 0Zm-10
0a2.5 2.5 0 1 1-5 0 2.5 2.5 0 0 1 5 0Z" />
          </svg>
          Fast Delivery
        </li>
      </ul>

      <div class="card-footer">
        <span class="price">${product.price}</span>
        <button class="add-btn btn-primary">
          <svg xmlns="http://www.w3.org/2000/svg" fill="none" viewBox="0 0 24 24">
            <path stroke="currentColor" stroke-linecap="round" stroke-
linejoin="round" stroke-width="2" d="M4 4h1.5L8 16m0 0h8m-8 0a2 2 0 1 0 0 4 2 2 0 0 0
0-4Zm8 0a2 2 0 1 0 0 4 2 2 0 0 0 0-4Zm.75-3H7.5M11 7H6.312M17 4v6m-3-3h6" />
            </svg>
            Add
          </button>
        </div>
      </div>
    `;

// Tombol Add memanggil callback dari app.js untuk memasukkan produk ke cart.
card.querySelector(".add-btn").addEventListener("click", function () {
  onAdd(product);
});

```



```

        container.appendChild(card);
    });
}

// Fungsi helper untuk menghasilkan icon bintang rating.
function getStarSVG() {
    return `

```

App.js

```

import { fetchProducts } from "../api/productApi.js";
import { getState, setState } from "../core/store.js";
import { ProductList } from "../components/ProductList.js";
import { Cart } from "../components/Cart.js";
import { Newsletter } from "../components/Newsletter.js";

// Fungsi untuk menambahkan produk ke keranjang dengan konsep immutability.
function addToCart(product) {
    const state = getState();

    // Spread operator membuat array cart baru tanpa mengubah state lama secara langsung.
    setState({
        cart: [...state.cart, product],
    });

    // Render ulang agar tampilan produk dan cart mengikuti state terbaru.
    render();
}

// Fungsi checkout mengosongkan keranjang setelah transaksi selesai.
function checkout() {
    setState({ cart: [] });
    alert("Thank you for your purchase!");
    render();
}

// Fungsi pusat untuk menggambar ulang komponen berdasarkan state aplikasi.
function render() {

```

```

    const state = getState();

    ProductList(state.products, addToCart);
    Cart(state.cart, checkout);
}

// Fungsi awal aplikasi: mengambil data produk lalu menampilkan UI.
async function init() {
  try {
    const data = await fetchProducts();

    // Simpan data produk ke state global sebelum komponen dirender.
    setState({ products: data });
    render();
    Newsletter();
  } catch (error) {
    // Menangani error jika proses pengambilan produk gagal.
    console.error("Failed to load products:", error);
  }
}

// Menjalankan aplikasi saat file app.js dimuat oleh browser.
init();

```

index.html

```

<!doctype html>
<html lang="id">

<head>
  <meta charset="UTF-8" />
  <title>Vanilla JS Web Store</title>
  <link rel="preconnect" href="https://fonts.googleapis.com">
  <link rel="preconnect" href="https://fonts.gstatic.com" crossorigin>

  <link
href="https://fonts.googleapis.com/css2?family=Inter:wght@400;500;600&family=Merriweather:ital,wght@0,400;0,700;1,400&display=swap" rel="stylesheet">
  <link rel="stylesheet" href="style.css" />

  <script src="https://code.jquery.com/jquery-3.7.1.min.js"></script>
  <script type="module" src="app.js" defer></script>
</head>

<body>
  <div id="app">
    <div class="container">

```

```

<div class="listing-header">
  <nav class="breadcrumb" aria-label="Breadcrumb">
    <a href="#">
      <svg xmlns="http://www.w3.org/2000/svg" fill="currentColor" viewBox="0 0
20 20">
        <path d="m19.707 9.293-2-2-7-7a1 1 0 0 0-1.414 0l-7 7-2 2a1 1 0 0 0
1.414 1.414L2
10.414V18a2 2 0 0 0 2 2h3a1 1 0 0 0 1-1v-4a1 1 0 0 1 1-1h2a1 1 0 0 1 1 1v4a1 1 0 0 0 1
1h3a2
2 0 0 0 2-2v-7.586l.293.293a1 1 0 0 0 1.414-1.414Z" />
      </svg>
      Home
    </a>
    <svg style="color: #9CA3AF;" xmlns="http://www.w3.org/2000/svg" fill="none"
viewBox="0 0 24 24">
      <path stroke="currentColor" stroke-linecap="round" stroke-linejoin="round"
stroke- width="2"
      d="m9 5 7 7-7 7" />
    </svg>
    <a href="#">Products</a>
    <svg style="color: #9CA3AF;" xmlns="http://www.w3.org/2000/svg" fill="none"
viewBox="0 0 24 24">
      <path stroke="currentColor" stroke-linecap="round" stroke-linejoin="round"
stroke- width="2"
      d="m9 5 7 7-7 7" />
    </svg>
    <span style="color: var(--text-muted);">Electronics</span>
  </nav>

  <div class="toolbar">
    <h2 class="page-title">Electronics</h2>
    <div class="toolbar-actions">
      <button class="btn-secondary">

        <svg xmlns="http://www.w3.org/2000/svg" fill="none" viewBox="0 0 24 24">
          <path stroke="currentColor" stroke-linecap="round" stroke-width="2"
d="M18.796
4H5.204a1 1 0 0 0-.753 1.659l5.302 6.058a1 1 0 0 1 .247.659v4.874a.5.5 0 0 0 .2.413
2.25a.5.5
0 0 0 .8-.4v-7.124a1 1 0 0 1 .247-.659l5.302-6.059c.566-.646.106-1.658-.753-1.658Z" />
          </svg>
          Filters
        </button>

```

```

        <button class="btn-secondary">
          <svg xmlns="http://www.w3.org/2000/svg" fill="none" viewBox="0 0 24 24">

            <path stroke="currentColor" stroke-linecap="round" stroke-
linejoin="round" stroke-width="2"
              d="M7 4v16M7 4l3 3M7 4 4 7m9-3h6l-6 6h6m-6.5 10 3.5-7 3.5 7M14 18h4"
            />

          </svg>
          Sort
        </button>
      </div>
    </div>
  </div>

  <div id="product-list" class="product-grid"></div>

  <div id="cart" class="cart-section"></div>
</div>

<script type="module" src="app.js"></script>
</body>

</html>

```

Styles.css

```

:root {
  --primary: #0d9488;
  --primary-hover: #0f766e;
  --primary-light: #ccfbf1;
  --bg-color: #f9fafb;
  --card-bg: #ffffff;
  --text-main: #111827;
  --text-muted: #6b7280;
  --border-color: #e5e7eb;
  --rating-star: #facc15;
  --font-serif: "Merriweather", serif;
  --font-sans: "Inter", sans-serif;
  --radius: 8px;
}

* {
  box-sizing: border-box;
  margin: 0;
}

```

```
padding: 0;
}

body {
  background: var(--bg-color);
  color: var(--text-main);
  font-family: var(--font-sans);
  line-height: 1.5;
  -webkit-font-smoothing: antialiased;
}

button,
input {
  font: inherit;
}

svg {
  height: 16px;
  width: 16px;
}

.container {
  margin: 0 auto;
  max-width: 1280px;
  padding: 32px 16px;
}

.flex-row,
.breadcrumb,
.breadcrumb a,
.toolbar,
.toolbar-actions,
.feature,
.rating-row,
.card-footer {
  align-items: center;
  display: flex;
}

.listing-header {
  margin-bottom: 32px;
}

.page-title {
  font-family: var(--font-serif);
  font-size: 1.875rem;
  font-weight: 700;
}
```

```
.eyebrow {
  color: var(--primary);
  font-size: 0.78rem;
  font-weight: 700;
  letter-spacing: 0;
  text-transform: uppercase;
}

.toolbar {
  flex-wrap: wrap;
  gap: 16px;
  justify-content: space-between;
  margin-top: 16px;
}

.toolbar-actions {
  gap: 12px;
}

.breadcrumb {
  color: var(--text-muted);
  font-size: 0.875rem;
  gap: 8px;
  margin-bottom: 12px;
}

.breadcrumb a {
  color: var(--text-muted);
  gap: 6px;
  text-decoration: none;
}

.breadcrumb a:hover {
  color: var(--primary);
}

.btn-primary,
.btn-secondary {
  align-items: center;
  border-radius: var(--radius);
  cursor: pointer;
  display: inline-flex;
  font-size: 0.875rem;
  font-weight: 500;
  gap: 8px;
  justify-content: center;
  transition: 0.2s;
```

```
}

.btn-secondary {
  background: #ffffff;
  border: 1px solid var(--border-color);
  color: var(--text-main);
  padding: 8px 12px;
}

.btn-secondary:hover {
  background: #3f4f66;
  color: var(--primary);
}

.btn-primary {
  background: var(--primary);
  border: none;
  color: #ffffff;
  padding: 10px 20px;
}

.btn-primary:hover {
  background: var(--primary-hover);
}

.btn-primary svg {
  height: 20px;
  width: 20px;
}

.layout {
  align-items: start;
  display: grid;
  gap: 24px;
  grid-template-columns: minmax(0, 1fr) 350px;
}

.product-grid {
  display: grid;
  gap: 24px;
  grid-template-columns: repeat(auto-fill, minmax(250px, 1fr));
  margin-bottom: 32px;
}

.product-card {
  background: var(--card-bg);
  border: 1px solid var(--border-color);
  border-radius: var(--radius);
```

```
    box-shadow: 0 1px 2px 0 rgba(0, 0, 0, 0.05);
    display: flex;
    flex-direction: column;
    overflow: hidden;
    transition: box-shadow 0.2s;
}

.product-card:hover {
    box-shadow: 0 4px 6px -1px rgba(0, 0, 0, 0.1);
}

.card-image {
    background: #ffffff;
    display: flex;
    height: 220px;
    justify-content: center;
    padding: 24px;
}

.card-image img {
    max-height: 100%;
    max-width: 100%;
    object-fit: contain;
}

.card-content {
    display: flex;
    flex: 1;
    flex-direction: column;
    padding: 24px;
}

.badge {
    background: var(--primary-light);
    border-radius: 4px;
    color: var(--primary);
    font-size: 0.75rem;
    font-weight: 600;
    margin-bottom: 12px;
    padding: 2px 10px;
    width: fit-content;
}

.card-title {
    color: var(--text-main);
    display: -webkit-box;
    font-size: 1.125rem;
    font-weight: 600;
```



```
line-height: 1.4;
margin-bottom: 8px;
overflow: hidden;
text-decoration: none;
-webkit-box-orient: vertical;
-webkit-line-clamp: 2;
line-clamp: 2;
}

.card-title:hover {
  text-decoration: underline;
}

.rating-row {
  gap: 8px;
  margin-bottom: 16px;
}

.stars {
  color: var(--rating-star);
  display: flex;
}

.rating-score {
  font-size: 0.875rem;
  font-weight: 600;
}

.rating-count,
.feature {
  color: var(--text-muted);
  font-size: 0.875rem;
}

.features-list {
  display: flex;
  gap: 12px;
  list-style: none;
  margin-bottom: 20px;
}

.feature {
  gap: 6px;
}

.card-footer {
  justify-content: space-between;
  margin-top: auto;
```

```
}

.price {
  font-size: 1.5rem;
  font-weight: 800;
}

.cart-section {
  background: #ffffff;
  border: 1px solid var(--border-color);
  border-radius: var(--radius);
  box-shadow: 0 1px 2px 0 rgba(0, 0, 0, 0.05);
  display: flex;
  flex-direction: column;
  padding: 24px;
  position: sticky;
  top: 24px;
}

.cart-section h2 {
  font-family: var(--font-serif);
  margin-bottom: 24px;
}

.empty-msg {
  color: var(--text-muted);
  font-style: italic;
}

.cart-list {
  display: flex;
  flex: 1;
  flex-direction: column;
  gap: 16px;
  margin-bottom: 24px;
  overflow-y: auto;
}

.cart-item {
  align-items: center;
  border-bottom: 1px solid var(--border-color);
  display: flex;
  gap: 12px;
  padding-bottom: 12px;
}

.cart-item img {
  border: 1px solid var(--border-color);
```

```
border-radius: 4px;
height: 50px;
object-fit: cover;
width: 50px;
}

.cart-details h4 {
display: -webkit-box;
font-size: 0.875rem;
font-weight: 600;
line-height: 1.35;
margin-bottom: 4px;
overflow: hidden;
-webkit-box-orient: vertical;
-webkit-line-clamp: 1;
line-clamp: 1;
}

.cart-details span {
color: var(--text-muted);
font-size: 0.875rem;
}

.cart-footer-summary {
border-top: 2px solid var(--border-color);
margin-top: auto;
padding-top: 20px;
}

.total-row {
align-items: center;
display: flex;
font-size: 1.125rem;
font-weight: 700;
justify-content: space-between;
margin-bottom: 16px;
}

.total-price {
color: var(--primary);
}

#checkout-btn {
width: 100%;
}

.newsletter-section {
background: var(--card-bg);
```

```
border-top: 1px solid var(--border-color);
margin-top: 64px;
padding: 48px 16px;
text-align: center;
}

.newsletter-content {
  margin: 0 auto;
  max-width: 500px;
}

.newsletter-content h3 {
  font-family: var(--font-serif);
  font-size: 1.5rem;
  margin-bottom: 8px;
}

.newsletter-content p {
  color: var(--text-muted);
  margin-bottom: 24px;
}

.newsletter-content form {
  display: flex;
  gap: 8px;
}

.newsletter-content input {
  border: 1px solid var(--border-color);
  border-radius: var(--radius);
  flex: 1;
  min-width: 0;
  padding: 10px;
}

#app {
  width: 100%;
}

@media (max-width: 900px) {
  .layout {
    grid-template-columns: 1fr;
  }

  .cart-section {
    position: static;
  }
}
```

```

@media (max-width: 560px) {
  .toolbar,
  .toolbar-actions,
  .newsletter-content form {
    align-items: stretch;
    flex-direction: column;
  }

  .btn-primary,
  .btn-secondary {
    width: 100%;
  }
}

```

Store.js

```

import { deepClone } from "../utils.js";

// State global menyimpan data utama aplikasi.
let state = {
  products: [],
  cart: [],
};

// getState mengembalikan salinan state agar data asli tidak berubah langsung.
export const getState = () => deepClone(state);

// setState memperbarui state dengan menggabungkan state lama dan data baru.
export const setState = (newState) => {
  // Spread operator membuat object baru sebagai penerapan immutability.
  const nextState = { ...state, ...newState };

  state = nextState;

  // Log digunakan untuk melihat perubahan state pada console browser.
  console.log("[Store] State updated:", state);
};

```

Utils.js

```

// State utama aplikasi disimpan di core agar mudah dipakai oleh file lain.
// deepClone membuat salinan data secara rekursif agar object/array asli tetap aman.
export function deepClone(obj) {
  // Nilai primitive seperti string, number, boolean, dan null langsung dikembalikan.
  if (obj === null || typeof obj !== "object") {
    return obj;
  }
}

```

```

}

// Jika data berupa Date, buat object Date baru dengan waktu yang sama.
if (obj instanceof Date) {
  return new Date(obj.getTime());
}

// Jika data berupa array, clone setiap item di dalamnya.
if (Array.isArray(obj)) {
  return obj.map((item) => deepClone(item));
}

// Jika data berupa object, clone setiap property miliknya.
const clonedObj = {};
for (const key in obj) {
  if (Object.prototype.hasOwnProperty.call(obj, key)) {
    clonedObj[key] = deepClone(obj[key]);
  }
}

return clonedObj;
}

```



